



Strategic Design & Panel Defense

Barangay Vehicle Management System

Every strategic feature — the gap it closes, the reasoning,
the workflow, and the dataflow — in one defensible reference.

Version 2.0.3 · Generated 18 July 2026

Backend feature suite: 72 automated tests passing

How to read this document

This is not a user manual — it is the "**why**" behind the system, written so you can defend every design decision to a panel. Each strategic feature is presented in five parts:

- **EXAMPLE** a concrete scenario that exposes the problem;
- **THE GAP** what the system used to fail to do;
- **THE SOLUTION** what was built and why;
- **WORKFLOW** who does what, step by step;
- **DATAFLOW** how the data actually moves through the system.

The one principle behind all of it. Every strategic feature is the same move: *turn a silent default into a conscious, reasoned, recorded decision by a person*. The system's job is to **surface the decision and record it** — never to make it silently or let something important happen (or fail to happen) unnoticed. Hold onto this sentence; it answers most panel questions.

1. Deferred sub-issues & decision-close

Example. An ambulance ticket has two problems; the brakes are fixed, but the water-pump seal is on back-order with no budget until Q4. The ticket can never reach "all done," so the vehicle would sit benched forever over one unfixable line item.

THE GAP

A sub-issue had only one exit — "Done." One item that couldn't be finished trapped the whole ticket, and the vehicle stayed out of service indefinitely.

THE SOLUTION

A second exit, **Deferred** — a recorded decision *not* to fix it now, with a mandatory reason. A ticket closes when every sub-issue is *resolved* (Done **or** Deferred). Closing with unfinished work ("decision-close") forces a **fit-for-service** answer — "No" keeps the vehicle out even though the ticket closed — and every deferred defect opens a **breadcrumb** issue report so it isn't forgotten.

WORKFLOW

1. Admin opens a sub-issue that can't proceed and clicks **Defer**, entering a reason.
2. System marks it Deferred and auto-opens a Pending follow-up issue report.
3. With all sub-issues Done/Deferred, Admin closes — answering "Is the vehicle fit to return to service?"
4. Fit = Yes → vehicle returns to Available; Fit = No → vehicle stays out.

```
Defer  Admin → PUT /tickets/{t}/sub-issues/{s}/defer {reason}
        → ticket_sub_issues.status = 'Deferred', deferred_reason/by/at set
        → createDeferralBreadcrumb() → vehicle_issue_reports (status 'Pending')

Close  MaintenanceTicket.isEligibleToClose() = every subIssue.isResolved()
        → closeTicket {returned_to_service} → vehicles.status set by fit-for-service
        → archiveCompleted() snapshot to ticket_archive_logs
```

2. Verification — the functional test

Example. A new water-pump seal is installed perfectly — the paperwork says "fixed." But nobody ran the pump. On the next fire, it leaks under pressure. "Looks done" was never the same as "actually works."

THE GAP

Verification could be satisfied by looking at the paperwork. It proved the work was *done*, not that the vehicle *works*.

THE SOLUTION

Verification is now a real **functional test**, run by the **Custodian** — deliberately a different role from the mechanic who did the work ("don't grade your own homework"). A **type-specific checklist** is marked Pass/Fail; any fail bounces the repair back to the mechanic; all-pass requires an **attestation** that the vehicle was actually operated.

WORKFLOW

1. Mechanic logs the repair → sub-issue goes *For Inspection*.
2. Custodian operates the vehicle against the checklist and marks each item.
3. Any Fail → *Rejected*, back to the mechanic. All Pass + attestation → *For Confirmation*.
4. Admin gives the final verdict → *Done*.

```
Custodian → PUT /tickets/{t}/sub-issues/{s}/verify {verdict, functional_test[], test_attested}
→ backend: Approved requires test_attested = true AND no failed item
→ ticket_sub_issues.functional_test (JSON), test_attested stored
→ pass: status 'For Confirmation' | fail: status 'Under Repair' (bounce)
```

3. Multi-role accounts

Example. A small barangay has one utility staffer, Juan, who is both the custodian and the mechanic. The system must let him do both jobs without pretending he is two people — and still keep the accountability trail.

THE GAP

Each account had exactly one role, so a real person wearing two hats couldn't be modelled without either a fake merged role or two logins.

THE SOLUTION

An account holds a **list** of roles (the existing three, un-merged). A *primary* role drives the portal; the sidebar shows the **union** of every role's modules; permission checks accept any hat held. Separation of duties is enforced by **role**, not by person, so every action still records which function was performed.

Concept	Mechanism
Which hats a person holds	<code>users.roles</code> (JSON), primary kept in <code>users.role</code>
Permission check	<code>hasRole()</code> / <code>hasAnyRole()</code>
Assignable-as lookups	<code>scopeHavingRole()</code> (matches primary OR list)
Which modules appear	<code>resolveModules()</code> unions all roles' menus

```
Login → returns user.roles = allRoles() | Add User → roles[] multi-select
Action by Juan → stored in the role-specific field (reported_by / assigned_mechanic_id /
verified_by)
→ the field that holds his id records WHICH hat he wore
```

Panel one-liner: "A role is a hat, not a headcount. One account can hold several; the system enforces separation by role and records which role acted — so the same system fits a two-person barangay and a twenty-person one with no changes."

4. Emergency Readiness (coverage by type)

Example. *The barangay owns one ambulance. It goes in for repair. The old dashboard said "1 vehicle under maintenance" — indistinguishable from a spare van being serviced. What it really meant: **zero medical-emergency coverage**.*

THE GAP

Availability was tracked per *vehicle*. But an emergency service is measured by *capability coverage* — for each function, is there a ready unit?

THE SOLUTION

Availability is re-aggregated **by vehicle type** — "Ambulance: 1 of 1 ready" — and the loss of the last ready unit of any type raises a hard **NO COVERAGE** alarm.

WORKFLOW

1. Any status change (ticket close, decommission, inspection) updates a vehicle's availability.
2. The dashboard recomputes coverage per type on every load.
3. A type dropping to zero ready units shows a red NO COVERAGE banner to every Admin.

```
GET /dashboard → fleetReadiness()  
  → group active vehicles by category  
  → ready = status 'Available'; no_coverage = (ready == 0 && total > 0)  
  → Readiness board + NO COVERAGE alarm
```

5. Preventive maintenance with teeth

Example. *The ambulance's brake servicing was due three weeks ago. The schedule was just a date on a calendar nobody read. Mid-transport, the brakes fail.*

THE GAP

A maintenance schedule had no consequences — overdue entries sat in a list, invisible until a breakdown.

THE SOLUTION

Overdue or due-within-7-days schedules surface on a dashboard **Preventive Watch**, colour-coded OVERDUE / DUE SOON — prevention made visible *before* failure.

```
GET /dashboard → preventiveWatch()  
  → vehicle_maintenance_schedules where status 'Scheduled' AND date ≤ today+7d  
  → state = (date < today ? 'overdue' : 'due_soon') → Preventive Watch list
```

6. Reliability & repeat-failure detection

Example. A rescue boat came in for engine trouble four times in five months. Each ticket was handled cleanly and closed — so nobody noticed it was the same boat, same failure, over and over.

THE GAP

Every ticket was an island. History was stored but never turned into insight, so a chronic (and dangerous) unit stayed hidden.

THE SOLUTION

A **recurring flag** on any ticket that repeats a recently-closed Main Issue on the same vehicle, plus a per-vehicle **reliability lens** (failures 6/12 mo, lifetime spend, avg days out, chronic warning).

```
Recurrence createTicket → count prior Closed tickets, same title, same vehicle, <90 days
→ maintenance_tickets.recurrence_count → "Recurring – Nth time" badge
Reliability GET /vehicles/{id}/reliability → aggregate tickets + maintenance_records
→ failures_6mo/12mo, total_spend, avg_days_out, chronic → profile panel
```

7. Decommission (end-of-life)

Example. A fire truck is written off after a crash. Someone just "archives" it — the same vague action used for a truck that's out temporarily. A year later nobody can tell if it was retired for good or hidden by mistake, and it still quietly inflates "we have 3 fire trucks."

THE GAP

The only way to remove a vehicle was a vague, reversible *Archive*. There was no honest, documented end-of-life distinct from a temporary hide.

THE SOLUTION

A distinct **Decommissioned** state with a mandatory reason. It leaves the readiness/coverage math for good, blocks new work, keeps full history, and can be **recommissioned** if mistaken. Open tickets must be settled first (use decision-close for an unfinishable one).

```
Admin → PUT /vehicles/{id}/decommission {reason}
→ blocked if any open ticket exists
→ vehicles.status 'Decommissioned' + decommission_reason/by/at
→ excluded from syncVehicleStatuses, readiness, lookups, ticket/issue creation
→ restore reverses (clears fields, status 'Available')
```

8. "Available" ≠ "Ready to respond" — readiness checks

Example. 1 AM, a call comes in. The dashboard says the ambulance is Available. The driver runs to it — near-empty fuel tank, no oxygen refilled, one dead siren speaker. The system said "ready." It wasn't.

THE GAP

"Available" meant only "no open repair." That is not the same as "can respond to an emergency right now." The system tracked "not broken" but not "mission-ready."

THE SOLUTION

A short, **type-specific readiness check** the custodian runs on a schedule (fuel, oxygen, siren, equipment). A vehicle is only **verified ready** if it passed a check within a **freshness window (24 h)**; older is *stale*, a failed check is *not ready*. This becomes the honest signal feeding the coverage board.

WORKFLOW

1. Custodian runs the readiness checklist (daily / pre-shift) on the vehicle.
2. All pass → vehicle reads *Ready to respond*; any fail → *Not ready*.
3. A check older than 24 h goes *stale* and lands on the dashboard Readiness Watch.
4. The coverage board shows "verified ready" alongside merely "available."

```
Custodian → POST /vehicles/{id}/readiness-check {checklist[], notes}
→ vehicle_readiness_checks (checklist JSON, all_passed, checked_at)
→ responseReadinessState(): Available + passed + <24h = 'ready';
                                older = 'stale'; failed = 'not_ready'; none = 'unchecked'
→ feeds readiness.verified_ready + readiness_watch + profile badge
```


9. Single-point-of-failure risk map

Example. *The barangay owns exactly one fire truck. Today it's running, so everything looks fine — until it breaks and everyone suddenly realises there was no backup all along.*

THE GAP

The readiness board alarms only *after* a unit is down (reactive). Nothing warned that a type had a single unit — a standing fragility — *while it was still healthy*.

THE SOLUTION

A proactive **fragility map**: any type with only one operational unit is flagged as a single point of failure, even while it's up — so the barangay can plan a backup before the crisis.

```
GET /dashboard → fragility()
  → group operational vehicles by type
  → single_point = (operational == 1); critical = (single && ready == 0)
  → Risk panel: "Only 1 X – no backup if it goes down"
```

10. Fleet-wide failure patterns

Example. *Across the fleet this year, 9 of 14 repairs were brake-related — but each was filed as its own ticket on its own vehicle, so nobody spotted that the flood-damaged roads are eating everyone's brakes.*

THE GAP

Reliability was per-vehicle. A *systemic* cause affecting many vehicles hid in plain sight because failures were never aggregated fleet-wide.

THE SOLUTION

A ranked "**what's breaking most**" view across the whole fleet over 12 months — surfacing a root cause to fix once instead of treating symptoms one ticket at a time.

```
GET /dashboard → failurePatterns()
  → vehicle_maintenance_records, last 12 months, grouped by maintenance_type
  → ranked by count → "What's Breaking Most" bar chart
```

11. Panel defense — anticipated questions

Q. "Isn't this just a vehicle tracker?"

A. No — it's a **readiness system**. It doesn't only ask "is this vehicle up?"; it asks "for each emergency function, do we have a *verified-ready* unit right now?" (\$4, \$8), warns where we're fragile before failure (\$9), and prevents breakdowns instead of only recording them (\$5).

Q. "It requires three roles — does that break a small barangay with two people?"

A. No. A role is a permission set, not a required employee. One account holds several roles; the workflow completes with one person and still records which role performed each step (§3). Hard-coding two roles would have been the real weakness — it wouldn't scale and would throw away the accountability control.

Q. "What if a repair legitimately needs weeks — won't things look stuck?"

A. That case has a home: **Deferral** (§1). A repair that must wait is consciously deferred with a reason and a breadcrumb — a recorded decision, not a silent stall.

Q. "What if the custodian who tests a repair is also the mechanic who did it?"

A. Separation is enforced by *role* — the functional test is the Custodian's gate, the repair is the Maintenance gate, the final verdict is the Admin's (§2). Even when one person holds two hats, the record shows which role signed each step.

Q. "Why keep a vehicle whose repairs cost more than it's worth?"

A. That's the barangay's call — the system's reliability lens (§6) surfaces the chronic unit and the spend, and decommission (§7) provides an honest, documented retirement. The tool informs the decision; it never forces it.

Q. "What is the design philosophy in one sentence?"

A. *Nothing important happens — or fails to happen — silently or unrecorded.* Every feature turns a silent default into a conscious, reasoned, recorded decision by a person.